

ATEM ISO ingest — near-real-time, Nextcloud-aware

Goal: get each theatre's ATEM Mini Extreme ISO recordings onto the mothership's Nextcloud as close to real time as the hardware allows, without corrupting anything and without the theatre's uplink falling permanently behind. The ATEM records up to 9 H.264 files (8 input ISOs + program, **up to 70 Mb/s per ISO file** per Blackmagic's spec); only the **program file plus the ISO inputs chosen in ATEM_ISO_INPUTS** (default: input 1, the camera) are pulled live — see Common theatre input map and Bandwidth. Everything else stays on the ATEM's SSD for the physical offload.

The two hardware facts that shape this

- **The ATEM has a built-in FTP server**, and files can be browsed/transferred over it *while recording is still in progress* — confirmed via multiple independent accounts (aaronparecki.com, Directory Opus forum). This is also the only mechanism available — the unit has no NDI output and its Ethernet port doesn't expose the drive over SMB/NFS, only FTP. **This holds for the original ATEM Mini Extreme ISO only.** The newer **ATEM Mini Extreme ISO G2** has a 10G Ethernet port and shares its CFexpress/USB recording media as a network disk (Blackmagic tech specs) — if the theatres have G2 units, a mount-based pull (like the VMix ingest) becomes possible. Confirm which model is actually deployed.
- **The files are very likely safe to read mid-write.** Blackmagic markets editing an ISO recording in DaVinci Resolve *before the event even finishes* — that only works if the container format is structured so a partial file is valid up to whatever's been flushed so far (in practice, this means fragmented MP4 — periodic `moof / mdat` boxes rather than one index at the very end). Treat this as **very likely true given the marketed capability, but worth a quick empirical check** (copy a file mid-recording, confirm `ffprobe` / a media player can open it) before relying on it operationally.

Common theatre input map

Decided 2026-10-06: **every one of the 12 ATEMs is patched identically**, so one ingest setting covers the whole fleet and an ISO file's input number means the same thing in every theatre.

ATEM input	Source	Pulled live?
1	Camera	Yes (default)
2	Presenter laptop 1 (PowerPoint Main)	Optional — only if router headroom allows (<code>ATEM_ISO_INPUTS=1,2</code>)
3	Laptop 2 (VT Main / second presenter)	Optional (<code>ATEM_ISO_INPUTS=1,2,3</code>)
4–8	Backups / spare	Never live — stays on the ATEM SSD
Program	The switched output	Always pulled

Why choose inputs at all. The ATEM's ISO recording is all-or-nothing per switcher — it can't disable individual inputs (an empty input just produces a near-empty file), and the ISO bitrate isn't user-settable; only the frame rate changes the target (~45 Mbps at 24/25/30p, ~70 at 50/60p, VBR — see Bandwidth). So the choice has to be made on the *pull* side: what crosses the theatre's uplink live, not what gets recorded. Everything is still recorded; everything not pulled live comes in via the physical DIT offload of the ATEM's SSD (or between sessions) — see `docs/live-editing.md` § physical media.

The program recording is different: it follows the ATEM's configurable Streaming/record quality setting rather than the fixed ISO rate. Set it so the program file is **~8–10 Mbps** (one third-party test measured ~4.8 Mbps at a lower setting) — the deployment runbook carries this as a per-unit step.

`ATEM_ISO_INPUTS` (comma-separated input numbers, default `1` ; empty = program only) is read by both ingest methods and set fleet-wide in `docker-compose.yml` / `.env.template` . `pull-iso.py` **walks each ATEM's recording folders recursively** (up to `<root>/<recording>/Video ISO Files/<file>`), pulls every `.mp4` / `.mov` that isn't an ISO file (i.e. the program recording), and pulls an ISO file only if the input number in its name is in `ATEM_ISO_INPUTS` . It recognises ISO files by `CAM <n>` in the file name — the naming as far as is known, still to confirm on a real unit (`docs/open-questions.md` #21); override the regex with `ATEM_ISO_INPUT_PATTERN` if it differs. The mirror keeps the ATEM's folder structure under `TheatreN/ISO/` . The `rcclone-mount-rsync/` alternative applies the same selection with rsync include/exclude filters built from the same variable (it matches `CAM <n>` by glob, not by `ATEM_ISO_INPUT_PATTERN`). Audio `.wav` source files and the `.drp` project are not pulled live by either method; they arrive with the physical offload.

Why not plain rsync, and why not plain rclone sync

Plain rsync can't connect at all — it needs SSH or an rsync daemon on the far end, and the ATEM only speaks FTP. There's no bridging that directly; the ATEM never exposes rsync or SSH.

A naive periodic `rcclone sync /whole-file re-copy` doesn't survive the bandwidth math. Real numbers: at the default live scope (camera ISO + program, ~45–80 Mbps, ~55 typical), a 3-hour session is already ~61–108 GB (~74 GB typical). Re-uploading the *entire current file* every 10 minutes stops fitting in that 10-minute window at the Slate AX's estimated ~150–250 Mbps Tailscale ceiling once the session is ~27–45 minutes old at the typical rate (~19 minutes at the 80 Mbps high case on a 150 Mbps router) — and that ignores everything else sharing the uplink — after which the sync falls permanently behind. Genuinely incremental transfer isn't a nice-to-have here, it's required.

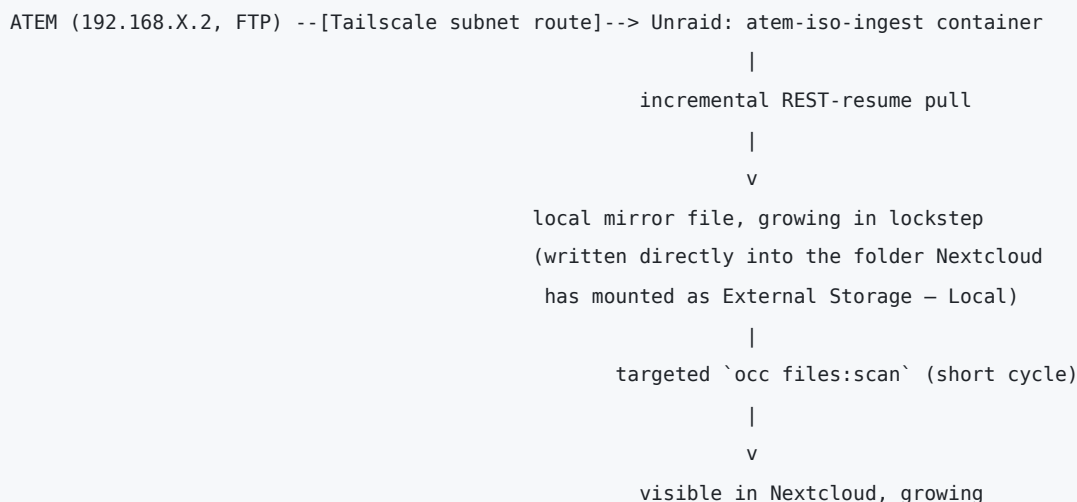
Two ways to get genuinely incremental transfer — both implemented, pick one (see `config/atem-iso-ingest/`):

	Mechanism	Why it's incremental
<code>pull-iso.py</code> (default)	FTP's own <code>REST <offset></code> command — the same "resume from byte offset" mechanism <code>curl --continue-at / lftp --continue / wget -c use</code>	Deterministic: explicitly requests only bytes past the last known offset. No cache layer involved.
<code>rcclone-mount-rsync/</code> (alternative)	<code>rcclone mount</code> presents the ATEM's FTP share as a local FUSE path; <code>rsync --append</code> (not <code>rsync</code> 's general checksum-diff algorithm) transfers only the tail past the destination's current size	Also deterministic — <code>--append</code> seeks straight to the destination's size rather than reading/hashing the whole file, so it doesn't depend on <code>rcclone</code> 's VFS cache behaving well for a growing remote file — a real concern for mount-based approaches that <code>--append</code> sidesteps entirely

Both are legitimate. `pull-iso.py` has fewer moving parts (no persistent FUSE mounts to supervise); `rclone-mount-rsync/` uses tools you may already operate day to day, at the cost of 12 FUSE mounts to keep healthy. Neither uses rsync's *general* block-checksum algorithm (which genuinely would require reading the whole file every pass) — that algorithm only saves network bytes when both ends speak the rsync wire protocol to each other, which isn't possible against an FTP-only source regardless of what sits in between.

Architecture

Centralized entirely on the Unraid server as its own container (`192.168.1.17`) — no software installed on any theatre laptop, consistent with how BirdDog Central/NDI Discovery Server/DERP are already consolidated there (the mothership VMix VM was removed — see `docs/open-questions.md` #10). Reaches each theatre's ATEM over Tailscale subnet routing (see `docs/tailscale.md` — this is exactly the "route between subnets where needed" case the subnet-router setup was built for). Described below for the default (`pull-iso.py`) — the `rclone-mount-rsync/` alternative differs only in the pull mechanism, writing into the same Nextcloud folders the same way.



1. **Pull step** (`atem-iso-ingest` container, new — see `config/atem-iso-ingest/`): for each theatre, on a schedule (e.g. every 60–120s), walk the ATEM's FTP recording folders recursively, and for each wanted media file (program + the inputs in `ATEM_ISO_INPUTS` — see Common theatre input map), `REST` -resume from the last recorded byte offset and append the new bytes to a local mirror file at the same relative path. A small state file tracks per-file offsets so a restart doesn't re-pull from scratch.
2. **No separate upload/chunking step.** The pull step writes directly into `/mnt/user/nextcloud-external/TheatreN/ISO/` — the same path mounted into Nextcloud as an External Storage (Local) folder for that theatre. There's no WebDAV re-upload of the growing file at all, so the "whole-file re-sync is too slow" problem simply doesn't apply on this side — it's a same-host filesystem write, not a WAN transfer.
3. **Nextcloud awareness:** a short-interval, *targeted* `occ files:scan --path=...` (not `--all`) picks up the new file size on Nextcloud's side. Because this only touches one theatre's folder and runs locally on the same box, it's cheap even at a 1-2 minute cadence — unlike scanning the whole Nextcloud tree, which would not be.

4. **On session end**, a final pull + final targeted scan catches the last bytes and whatever moov/index finalization is *appended* when the ATEM's recording actually stops. **It does not catch finalization that rewrites earlier bytes in place** — MP4 muxers commonly patch a box size or header near the start of the file when recording stops (OBS's hybrid-MP4 "soft remux" is a documented example: obsproject.com). Both ingest methods are append-only and would leave such a mirror with stale header bytes. Whether the ATEM does this is unconfirmed; until it is, treat the mirror as "review copy" and either re-pull the finished file in full or run a checksum comparison against the source before treating it as final (see Open items).

Second write destination for live editing. The pull step also writes the same growing mirror file to a second mount — the edit suite's dedicated NAS (192.168.22.x), not just Nextcloud's External Storage. One read off the ATEM, two writes, not a separate re-sync reading Nextcloud's copy back out. See docs/live-editing.md for the full editing subsystem this feeds — not yet reflected in config/atem-iso-ingest/pull-iso.py itself, which currently only writes the one Nextcloud destination; adding the second write path is a real code change, tracked in docs/open-questions.md .

Bandwidth

Resolved by decision (2026-10-06): the old ~10 Mbps-per-ISO working figure was wrong, and the design now pulls only a chosen subset live. Blackmagic's tech specs say the ISO inputs are recorded as "H.264 .mp4 files at up to 70Mb/s quality" (blackmagicdesign.com). The ISO bitrate is not user-settable — only frame rate changes the target, ~45 Mbps at 24/25/30p and ~70 at 50/60p, VBR (Blackmagic forum; the feature request for a setting is still open). User reports put the *average* at ~30–45 Mbps per ISO (e.g. ~15 GB/h per source ≈ 33 Mbps, forum); older firmware showed 90–120 Mbps, with a silent drop around firmware 9.6.x (forum). Only the **program** file follows the configurable quality setting, so ~10 Mbps is realistic for it alone. Pulling all 9 files live (~370–570 Mbps per theatre at 45–70 Mbps per ISO) is therefore off the table on the chosen router; the fix is the common input map plus ATEM_ISO_INPUTS . Still to measure on a real unit: the actual camera and laptop ISO bitrates, and the program file at the chosen quality setting (docs/open-questions.md #0).

Planning figures (low / typical / high): camera ISO **35 / 45 / 70 Mbps** (user reports; 70 = vendor cap), laptop (slides) ISO **15 / 25 / 40 Mbps** — **an unmeasured estimate**, one measured file pins it, program **10 Mbps**. Ingest in isolation, per theatre, including the ~4% WireGuard overhead:

Live scope (ATEM_ISO_INPUTS)	Payload	Ingest incl. WireGuard	vs. Slate AX (~150–250 Mbps est.)
Camera + program (1 , default)	45 / 55 / 80 Mbps	47 / 57 / 83 Mbps	comfortable
Camera + 1 laptop + program (1,2)	60 / 80 / 120 Mbps	62 / 83 / 125 Mbps	fits
Camera + 2 laptops + program (1,2,3)	75 / 105 / 160 Mbps	78 / 109 / 166 Mbps	fits at typical; tight at high
4 ISOs (2 cameras + 2 laptops) + program	110 / 150 / 230 Mbps	114 / 156 / 239 Mbps	marginal

(Per-theatre ATEM output at the default scope is ~20 / 25 / 36 GB per hour; at camera + 2 laptops ~34 / 47 / 72 GB/h.)

This rides the theatre's uplink in the *opposite direction* from the incoming SRT feed (~5.8 Mbps baseline / ~10.4 peak, see docs/bandwidth-analysis.md) — if the link is genuinely full-duplex-capable at its rated throughput, those two shouldn't compete much. But the ingest does **not** have the upstream direction to itself: the theatre's ATEM Overseer monitoring stream and Flock SRT preview (~10.4 Mbps each) run

theatre → mothership alongside it, plus bursty rclone (~5), a fixed ~26 Mbps. Total upstream per router is then ~73 / 83 / 109 Mbps at the default scope and ~104 / 135 / 192 Mbps at camera + 2 laptops. The authoritative per-router model is in `docs/bandwidth-analysis.md`; the table above covers the ingest in isolation only. The Slate AX's ~150–250 Mbps is an **estimate** extrapolated from a Beryl AX Tailscale measurement, not a Slate AX benchmark — benchmark one unit bidirectionally before relying on it (`docs/open-questions.md` #19). (The previous router, the A-1300, was dropped on 2026-10-06 because its ~170 Mbps figure is GL.iNet's *kernel* WireGuard number; Tailscale on its 32-bit CPU is estimated at only ~30–70 Mbps combined.) `ATEM_ISO_INPUTS` is the fleet-wide default in both ingest methods, and `ATEM_ISO_INPUTS_T<n>` overrides it for one theatre (e.g. `ATEM_ISO_INPUTS_T3=1,2` adds Theatre 3's main laptop only). Use the override rather than raising the default: at camera + 2 laptops in every theatre, the fleet's inbound total can exceed the mothership's 2× 1GbE bond at the high end (`docs/server-specification.md` § NIC).

Master vs. mirror

Frame this as: **the ATEM's own SSD remains the authoritative final master** (complete, guaranteed-valid once recording stops) — pulled in full at the end of each session/day regardless. **Nextcloud holds a near-real-time, continuously-updated mirror** for early review/rough-cut purposes during the event. That framing resolves the tension between "want it fast" and "want it guaranteed correct": you get both, from two different consumers of the same underlying data.

Nextcloud versioning caveat

Nextcloud's versioning app may retain snapshots on repeated changes to the same file path. Configure version retention/expiry for the ISO folders (or disable versioning there specifically) so a multi-hour growing recording doesn't accumulate many near-duplicate historical versions before Nextcloud's own expiry curve catches up. Not a blocker, just worth tuning once this is running for real.

Open items

- **Measure real file bitrates on a unit** — the live scope is decided (see Common theatre input map), but the planning figures aren't measured: record a camera ISO and a laptop/slides ISO at the event's frame rate and firmware, and the program file at the chosen quality setting, then `ffprobe` them (or file size ÷ duration). The laptop figure (15 / 25 / 40 Mbps) is a pure estimate; one file pins it. Tracked as `docs/open-questions.md` #0.
- **Confirm the ATEM model (original vs. G2)** — the G2 exposes its media as a network disk, which changes which pull mechanism is the natural fit (see the hardware facts at the top).
- **Test the recursive walk against a real ATEM.** (Previously: "`pull-iso.py` doesn't recurse" — fixed.) Blackmagic's ISO recordings are written as a *folder* per recording (program file, a `Video ISO Files` subfolder, an `Audio Source Files` subfolder of `.wav`s, and a `.drp` Resolve project); `pull-iso.py` now walks that tree (MLSD, falling back to NLST + a `CWD` probe, since the ATEM's FTP command set is unconfirmed) up to three levels deep and keeps the folder structure in the mirror. Untested against real hardware — confirm the walk finds the files and the depth limit is enough

(docs/open-questions.md #22). `.wav / .drp` are deliberately not pulled live by either method; they come with the physical offload.

- **Decide the end-of-session integrity step.** Both methods are append-only (see step 4 under Architecture). For `rclone-mount-rsync/`, `VERIFY=1` now does a full `--checksum` comparison and repairs any differing blocks in place; note that this reads every file in full through the FTP mount — i.e. a full re-download over the theatre's uplink — so run it after the session, not during. `pull-iso.py` has no equivalent yet.
- **Confirm partial-file playability empirically** — copy a file mid-recording, check it opens/scrubs correctly, before treating the "safe to read mid-write" assumption as settled rather than "very likely."
- **Tune the scan interval and pull interval** against real session lengths and available Unraid CPU/disk headroom, once the server's full spec is known (see docs/open-questions.md).
- **Confirm the ATEM's actual FTP file/folder naming**, in particular that ISO files carry `CAM <n>` in their names — input selection depends on it. If a real unit names them differently, set `ATEM_ISO_INPUT_PATTERN` for `pull-iso.py` and edit `build_filters` in `rclone-mount-rsync/mount-and-sync.sh` ; a file that doesn't match is treated as a program recording and pulled, so a mismatch fails toward pulling *everything*, not nothing (docs/open-questions.md #21).
- **If using the `rclone-mount-rsync/` alternative**, test `rsync --append` against a genuinely growing file on a real ATEM before trusting it operationally — confirm it only transfers the new tail each pass (e.g. watch network throughput or `--itemize-changes` output), and budget for supervising 12 persistent FUSE mounts (auto-remount on failure), which `pull-iso.py` doesn't need at all. Also confirm new bytes show up promptly: FTP has no change notification, so a growing file's size on the mount only refreshes when rclone's directory cache expires (default 5 minutes — the script now sets `--dir-cache-time 30s` ; rclone mount docs).